

제안해 드린 목차와 이전 흐름을 바탕으로, 딥러닝 연산의 병목을 해결하고 운영 안정성을 확보하기 위한 백엔드 엔지니어링 과정을 다루는 ****제5장. 비동기 백엔드 배포 및 MLOps 파이프라인****을 작성해 드립니다.

제5장. 비동기 백엔드 배포 및 MLOps 파이프라인

본 장에서는 제3장과 제4장에서 구축하고 최적화한 이기종 멀티모달 AI 모델(Animated Drawings, CogVideoX, GPT-SoVITS, MusicGen 등)을 실제 서비스 가능한 웹 애플리케이션으로 통합하는 과정을 서술한다. 극심한 GPU 메모리 점유율과 긴 연산 시간(수십 초~수 분)을 요구하는 생성형 AI의 특성상, 일반적인 동기(Synchronous) 방식의 웹 서버 아키텍처는 즉각적인 응답 지연(Timeout)과 서버 마비(Crash)를 초래한다. 이를 해결하기 위해 K-Ride 시스템에 도입된 비동기 메시지 큐 구조와, 생성된 결과물 및 실험 데이터를 체계적으로 추적하기 위한 MLOps 연동 전략을 분석한다.

5.1. FastAPI와 Celery/Redis 기반 비동기 미디어 처리

K-Ride 시스템의 백엔드는 고성능 비동기 웹 프레임워크인 **FastAPI**를 기반으로 설계되었으며, 클라우드 환경(Kaggle 등) 내부의 포트를 외부 퍼블릭 URL로 개방하기 위해 **zrok 터널링(Tunneling)** 기술을 적용하여 프론트엔드와 통신한다 1. 그러나 무거운 AI 연산의 블로킹(Blocking)을 방지하기 위해 핵심 렌더링 로직은 메인 API 서버와 완전히 분리된 비동기 작업 큐로 위임된다.

1) 무거운 AI 연산의 블로킹 방지를 위한 메시지 큐 구조 도입사용자로부터 이미지와 텍스트(일정)가 접수되면 FastAPI 메인 서버는 즉시 고유한 작업 식별자(job_id)를 발급하고 응답을 반환한다. 실제 미디어 생성 작업은 Redis를 메시지 브로커(Broker)로 삼아 큐(Queue)에 적재되며, 독립된 가상환경 샌드박스에서 대기 중인 ****Celery 워커(Worker)****가 이를 수신하여 백그라운드 연산을 시작한다 2-4. 특히 파이프라인의 붕괴를 막기 위해 Celery 워커 내부에는 '**3중 생존 방어막(Error Migration)**' 로직이 적용되어 있다. 1단계로 정상적인 비디오 렌더링을 시도하고, 메모리 한계로 실패할 경우 2단계로 개별 프레임(PNG)을 디스크에 우선 추출한 뒤 FFmpeg로 강제 조립하며, 이마저 실패할 경우 3단계로 5초짜리 더미(Dummy) 영상을 반환하여 전체 파이프라인(TTS, BGM 믹싱 등)이 중단되는 것을 원천 차단한다 5.

2) FFmpeg를 이용한 비디오, 오디오(TTS), 배경음악(BGM) 볼륨 믹싱 및 정규화비동기 워커를 통해 병렬로 생성된 ① 비디오(MP4 또는 GIF), ② 한국어 내레이션(TTS WAV), ③ 배경음악(MusicGen BGM WAV)은 최종적으로 단일 미디어 컨테이너로 합성된다. 이 믹싱 과정은 파이썬의 과부하를 피하기 위해 OS 레벨의 FFmpeg 엔진으로 오프로딩된다 5, 6. 단순한 파일 병합을 넘어, 브이로그로서의 시청각적 품질을 높이기 위해 복합 필터(-filter_complex)가 적용된다. AI가 생성한 BGM이 내레이션의 전달력을 해치지 않도록 BGM의 볼륨을 -10dB에서 -18dB 수준으로 감쇠(Ducking)시키는 오디오 정규화 필터([2:a]volume=-18dB[bgm_low])를 거친다 5, 7. 이후 오디오 믹스 필터(amix)를 통해 두 소리를 합성하며, -shortest 옵션을 인자로 주어 비디오나 오디오 중 가장 짧은 트랙의 길이에 맞춰 영상 재생이 자연스럽게 종료되도록 동기화한다 5, 8.

5.2. MLflow 및 DagsHub 연동을 통한 실험 추적

수많은 하이퍼파라미터 조율과 프롬프트 엔지니어링이 수반되는 멀티모달 파이프라인의 특성상, 어떤 입력값이 어떤 결과물을 냈는지 체계적으로 기록하는 MLOps 체계가 필수적이다. K-Ride 시스템은 로컬 환경에 의존하지 않고 클라우드 기반 원격 추적 저장소인 **DagsHub**와 **MLflow**를 통합하여 파이프라인 가시성을 확보했다.

1) 파라미터 및 프롬프트 로깅 (Log Params)작업이 실행될 때마다 MLflow의 `start_run()` 컨텍스트가 개방되며, 사용자가 입력한 프롬프트(`prompt_text`, `bgm_prompt`), 라우팅된 타겟 모델(`target_model`), 폴백(Fallback) 사용 여부 등 렌더링의 뼈대가 되는 메타데이터가 파라미터로 기록된다 1, 9-11. 이를 통해 이후 결과물의 품질을 분석할 때 어떠한 조건 분기에서 연산이 수행되었는지 즉시 역추적할 수 있다.

2) 생성 소요 시간 및 시스템 지표 로깅 (Log Metrics)클라우드 자원(T4 GPU 등)의 효율성을 평가하기 위해 각 모델별 처리 시간 지표를 분리하여 측정한다. 애니메이션 렌더링 소요 시간(`animation_seconds`), TTS 생성 시간(`tts_seconds`), BGM 합성 및 FFmpeg 믹싱 시간(`ffmpeg_seconds`)이 각각 수집되며, 워커의 비정상 종료를 방어한 결과 상태(Success: 1, Fail: 0) 및 리턴 코드(`return_code`)가 지표(Metric)로 기록되어 병목 구간 파악에 활용된다 1, 5, 12.

3) 최종 아티팩트(MP4) 및 에러 로그 기록 체계 (Log Artifacts)중간 과정에서 생성되는 수백 장의 개별 프레임은 클라우드 스토리지 낭비를 초래하므로 기록하지 않는다. 대신, 파이프라인의 최종 산출물인 브이로그 비디오(`final_video.mp4`), 전체 과정을 요약한 JSON 파일(`render_summary.json`), 그리고 OOM이나 패키지 충돌 발생 시 원인 분석을 위한 표준 에러 텍스트(`render_stderr.txt`)만을 선별하여 MLflow의 아티팩트(Artifact)로 DagsHub 서버에 업로드한다 1, 5. 이 기록 체계 덕분에 런타임 세션이 종료(Kill)되더라도 이전의 렌더링 결과물과 실패 로그를 중앙 대시보드에서 안전하게 조회하고 다운로드할 수 있다.

💡 다음 진행 제안이로써 **K-Ride AI** 프로젝트의 가장 방대하고 깊이 있는 기술적 분석(1장~5장)이 모두 완료되었습니다.마지막으로 이 긴 여정을 요약하고, 시스템의 현주소와 향후 클라우드(Supabase 등) 확장 계획을 담은 ****제6장. 결론 및 향후 과제****를 마무리하여 전체 20장 분량의 기술 레포트를 완성해 드릴까요?